

Resumo tecnico do banco SQLite atual

Documento de apoio para projetar o PostgreSQL no Supabase. Nenhuma alteracao foi feita no SQLite, no codigo ou no Supabase.

Arquivo: /workspace/repo/data/waleska.db (81920 bytes, UTF-8). Backend ativo: Flask + sqlite3 em app.py. DB_BACKEND=supabase existe no .env.example, mas nao ha cliente Supabase implementado. O frontend nao acessa o banco.

1. Tabelas existentes

admins (1)	Login do p
services (6)	Catalogo agenda e
registrations (7, IDs 3-9)	Pacientes status, for
waitlist (2)	Lista de e
duplas (1)	Duplas d appointme
appointments (2)	Agenda. C service_id
payments (5)	Financeiro agendame
evaluations (0)	Avaliacao
evolutions (0)	Evolucao
exercises (4)	Biblioteca
documents (0)	Document
settings (9)	Opcoes d

Nao migrar: sqlite_sequence (interno do SQLite). Nao ha FOREIGN KEY declarada. Relacionamentos sao IDs inteiros + nomes copiados em texto.

2. Essenciais para o funcionamento atual

Sem estas o aplicativo quebra: admins, services, registrations, waitlist, duplas, appointments, payments, settings.
Estrutura em uso no codigo, mesmo vazias: evaluations, evolutions, exercises, documents.

3. Dados reais a preservar

Sim. Nao e apenas seed. O banco local em uso contem:
- 1 admin (admin@waleskapaula.com.br, hash scrypt Werkzeug)

- 6 servicos com precos reais
- 7 pacientes, 2 na espera, 1 dupla, 2 agendamentos, 5 pagamentos (1 pago)
- 9 settings (dias/frequencias/motivos + WhatsApp, endereco, horarios)
- 4 exercicios da biblioteca

Ha cadastros de teste, mas este e o banco da plataforma local. Preservar tudo se for a base da migracao.

4. Login / autenticacao de administrador

Auth propria, sem Supabase Auth. POST /api/admin/login valida e-mail + senha na tabela admins. Hash Werkzeug scrypt em password_hash. Sessao Flask: session[admin_id]. /admin redireciona para /admin-login; APIs usam @login_required. O frontend nao acessa o banco.

5. Riscos na ida para PostgreSQL/Supabase

- PKs inteiras com buracos (registrations 3-9, payments 2-6). O Flask usa esses IDs. UUID quebra a API.
- Sem FK real; nomes denormalizados (patient_name, servico, service_name).
- JSON em TEXT: dias, motivos, duplas.paciente1/paciente2, settings.valor.
- Datas em TEXT, nao date/timestamptz. Booleanos 0/1 em services.lista_espera e ativo.
- payments.patient_id e registration_id hoje sao o mesmo ID.
- duplas.criadaEm (nao criadoEm).
- O rascunho supabase/schema.sql NAO espelha o SQLite (UUIDs, staff_users, patients, sem payments/exercises). Nao usar como mapeamento 1:1.

6. IDs que devem permanecer iguais

Obrigatorio manter os IDs atuais (incluindo buracos) em: admins.id, services.id, registrations.id (e o patient_id de agenda, financeiro, espera e prontuario), waitlist.id (esta no JSON das duplas), duplas.id, appointments.id, payments.id, exercises.id. settings usa PK textual (chave); preservar as chaves, nao um id numerico. evaluations, evolutions e documents estao vazias; ainda assim criar as tabelas com PK integer identity, alinhadas ao Flask.

Relatorio somente leitura. Nenhuma tabela foi criada no Supabase. SQLite intacto. Sem commit ou push.